# Quiz: Sorting

**Total Score:** $/2^2$

**Printed Name:**

---

**Quiz rules:**

1. You MAY use any printed or handwritten notes.
2. You MAY NOT use a computer or any other electronic device.

**Problem 1.** Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def foo(xs):
    def go(i, acc):
        if i == len(xs):
            return acc
        if xs[i] % 2 == 0:
            return go(i+1, acc - xs[i])
        return go(i+1, acc + xs[i])
    return go(0, 0)
try:
    print('foo([1, 2, 3, 4, 5])=',foo([1, 2, 3, 4, 5]))
except RuntimeError:
    print('StackOverflow')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

**Problem 2.** Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
xs = ['kiwi', 'fig', 'cherry', 'banana', 'date', 'apple']
zs = list(sorted(xs, key=lambda w: len(w)))
print('zs=', zs)
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 11 / 16 = 0.69

**Problem 3.** Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def merge_sorted(xs):
    if len(xs) <= 1:
        return xs
    mid = len(xs) - 1
    return merged(merge_sorted(xs[:mid]), merge_sorted(xs[mid:]))

def merged(xs, ys):
    out = []
    i = 0
    j = 0
    while i < len(xs) and j < len(ys):
        if xs[i] <= ys[j]:
            out.append(xs[i])
            i += 1
        else:
            out.append(ys[j])
            j += 1
    out += xs[i:]
    out += ys[j:]
    return out

xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
try:
    zs = merge_sorted(xs)
    print('zs=', zs)
except RuntimeError:
    print('StackOverflow')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $12 / 16 = 0.75$

**Problem 4.** Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def quick_sorted(xs):
    if len(xs) <= 1:
        return xs
    mid = len(xs) // 2
    pivot = xs[mid]
    smaller = [x for x in xs if x <= pivot]
    larger  = [x for x in xs if x >= pivot]
    return quick_sorted(smaller) + quick_sorted(larger)

xs = [3, 7, 1, 8, 5, 10, 2, 9, 4, 6]
try:
    zs = quick_sorted(xs)
    print('zs=', zs)
except RuntimeError:
    print('StackOverflow')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

## LLM Model Performance

